Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 225 (2023) 3660–3669

Procedia Computer Science

www.elsevier.com/locate/procedia

27th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2023)

Job Seeker Recommendation for Employers: A Graph-Based Recommendation Approach Using Node Embedding

Fatma Gizem Çallı[a,*], Günce Keziban Orman[b]

[a]Kariyer.net Electronic Publishing and Communication Services Inc., Umraniye, Istanbul and 34768, Turkey
[b]Galatasaray University, Ortakoy, Istanbul and 34349, Turkey

Abstract

Through the growth and development of online recruitment platforms, thousands of individuals can see vacancies promoted by companies. The fact that hundreds of job seekers are applying for a single position prolongs the hiring process for companies and increases the need for human resources to review these applications. In this study, a graph-based recommendation system is suggested as a solution to the issue that many companies are now experiencing. Node embedding and feature representation methods are explored and the link prediction successes of these methods are examined, by associating the candidates that applied for the same jobs in a graph structure.

© 2023 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 27th International Conference on Knowledge Based and Intelligent Information and Engineering Systems

Keywords: recommendation systems; node embedding; job seeker recommendation; social network analysis;

1. Introduction

In comparison to previous decades, it is now easier for candidates to find employment thanks to the development of online employee recruitment platforms. However, there are some unpredicted drawbacks to this ease as well. Due to the ease of online application to vacancies, job seekers can make applications haphazardly, and this situation poses a problem for companies [1]. Since job advertisements attract both individuals who may be eligible for the position, as well as those who are not employers, find it hard to identify the ideal candidate [2]. As a result, hiring procedures take longer than expected, which delays the recruitment of employees that the company seeks, hiring more Human Resources (HR) specialists becomes a necessity, and this situation causes financial losses for companies [3].

* Fatma Gizem Çallı. Tel.: +90-545-209-2129 ; fax: +90-212-326-9175.
E-mail address: gizem.calli@kariyer.net

1877-0509 © 2023 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 27th International Conference on Knowledge Based and Intelligent Information and Engineering Systems
10.1016/j.procs.2023.10.361

There are many different needs for online recruitment platforms, such as identifying the entities of resumes in various formats [33], recommending job seekers on positions that would be a suitable match for their resumes [32], or recommending a career path to new recruits. The leading online job recruitment platform in Turkey, Kariyer.net, provides solutions for these positions that are required in online job recruitment systems by employing different approaches. These solutions assist to advance the establishment of more effective and faster recruitment processes. A comprehensive study was also conducted to come up with a solution for the issue of recommending a candidate to the employer, which is highlighted within the scope of this article.

To solve the specified employee recruitment problem which is experienced by many companies, a graph-based recommendation system is proposed in this paper that recommends potential employees who exhibit similar job application behavior (application for the same jobs) to the applicant that employers are interested in. Graph-based solutions for recommendation systems have taken off recently because they have the potential to overcome well-known issues of recommendation engines such as cold start or data sparsity [4][5][6]. This system we propose in this work premise is that applicants with comparable professional backgrounds and job interests will submit applications for comparable positions. Following this purpose, Kariyer.Net A.Ş. provided the employment application data.

The primary goal of this study is to use job seekers' application behavior to suggest qualified candidates for open positions. The activity of job seekers' applications for open positions is recorded in our system. Based on job seekers' application behavior for the same positions, we can identify candidates who are similar to each other. In the suggested approach, our system makes new candidate suggestions who have not yet applied for the related position but have similar job application behavior to the related applicant while an employer examines an applicant's resume.

The main contributions of this work can be listed as follows:

- A novel candidate recommendation system for employers is proposed. Since most of the existing work is concentrated on proposing jobs for candidates, this work is dedicated to facilitating the candidate search for employers.
- Three foremost graph algorithms, node2vec, GraphSAGE, and Graph Convolutional Network (GCN) are used in the proposed system to overcome the common issues of recommendations, such as the sparsity of the data and the cold start.
- The experiments are performed on a large real-world dataset. The dataset reflects the real actions of job candidates and employers. It is not pruned or reduced.

In the rest of the article, we explain the related works in Section 2. Then, we explain the method, the experiments, the results, and the conclusion, respectively.

2. Related Works

In the majority of industries, companies, and candidates now use online employee recruitment platforms for their recruitment operations [3]. By this means, employers and job seekers can fulfill their recruitment and job search needs in a shorter time by spending less labor and time [2][3]. These platforms have been able to gather more data over time due to the growing popularity of online recruitment by both employers and job seekers [7]. In-depth analyses of employer and job seeker preferences were made possible because of the large-scale data pools that had been produced [8].

In order to boost user satisfaction and keep users on the platform, many websites started to employ recommendation systems. [17]. There are numerous cases where products that may be of interest to us are suggested based on our current needs and preferences when watching movies or shopping online [9][10][11][12].

Recommender systems are also widely used in job search platforms [13] [14][15]. In the recommendation systems works, we encounter conventional content-based filtering. They use semantic similarity to match candidates with vacancies with similar features, producing good outcomes but being limited in their ability to make recommendations to candidates who had not provided enough information since they were limited to the information entered by the user. In the worst case, this scenario is called the cold start problem, which addresses the lack of recommendations for newcomers to the system. The collaborative filtering (CF) method can cross-suggest users based on shared user behaviors, but a ranking requirement for the recommendation is not applicable in some of the cases [16], and with the

increasing number of items and users, CF suffers from sparsity [19][20]. In [21], it is proposed a graph-based structure to solve the sparsity problem in collaborative filtering.

Aggarwal et al. explore the success of inductive learning methods in graph-based job recommendation and explains the advantages and limitations of each of the models in [22]. It is stated that the GraphSAGE algorithm produced a higher F1 score over others by the author. According to [23], Graph Convolutional Network (GCN), which incorporates both node features and global information, is more successful at link prediction than previously employed graph-based recommendation systems since they neglected the global features. A resume recommendation system using a vector space model to identify the most qualified candidates was implemented in [24] by addressing the same problem for the recruiter as in this study.

Previous studies on recruitment recommendations have concentrated on concepts such as collaborative filtering, content-based filtering, and graph-based CF. However, these approaches were mostly limited to content data and were insufficient to address the problems of the employers. At this point, it is apparent that graph-based systems that learn from user behaviors and use state-of-the-art methods will have a favorable impact on the literature.

3. Methods

In this section, we represented the detailed architecture of our Job Seeker Recommendation System, see Figure 1. We explicitly described our data preparation, node embedding, model training, and classification steps, respectively. The experimental setup of the methods is explained in Section 4.

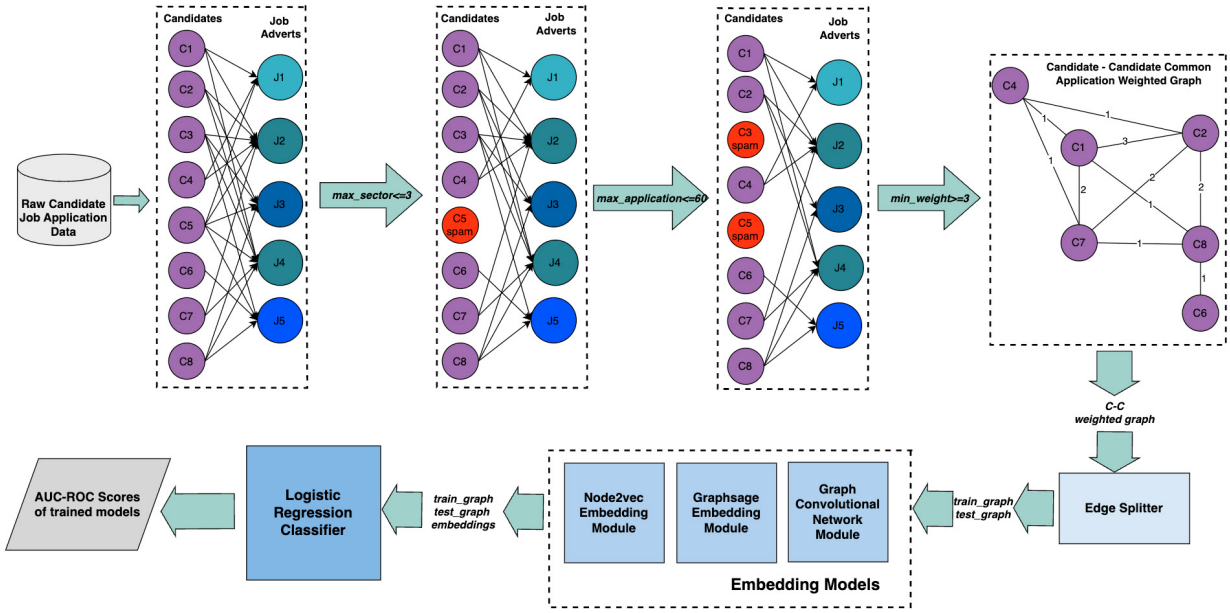

Fig. 1. System architechture of the proposed candidate Recommendation System.

3.1. Data Characteristics and Preparation

In this study, a real-world data set with users' job application activities was used. The data set is extracted from a job recruitment organization, which is the foremost online job recruitment platform in Turkey. Both job seekers and employers use this platform quite frequently since it provides the most recent job advertisements in the entire country. We concentrate on recommending new job seekers to employers with the aim of shortening the candidate search time and finding the punctual candidate for the true positions. Each job application made by candidates within the system over a particular period was collected as a raw data set along with their unique candidate and job posting ids. 30-days

(1 month), 21-days (3 weeks), and 14-days (2 weeks) period of job application data sets were extracted; however, only the 10 days of application data could be used for the implementation of this study since there are not enough hardware features to handle such high dimensional data. The 10 days of job application data consist of 2775243 job application behavior from 56843 different vacancies and 473093 unique candidates. In order to eliminate spam users who display random job application behavior, two thresholds were employed based on the pre-determined [31] and tagged sector class of the job postings (max_sector ≤ 3) and the average daily application amount of the users (max_application≤60). After applying the job sector number and job application number thresholds, the workload of the system for job applications is lightened almost in half, the total number of job applications, different vacancies, and unique job seekers is reduced to 1224527, 54329, and 388742 respectively with the difference between the unique candidate numbers signifying spam users. The change in the unique job and candidate numbers is shown in Table 1.

Table 1. Unique Job and candidate numbers represented on dataset after the related threshold operation.

	original data set	max_sector≤3	max_application≤60
#Unique Jobs	56843	54370	54329
#Unique candidates	473588	388969	388742

In the sequel, a candidate-job bipartite graph was extracted, where each user was paired with the vacancies they applied for. A weighted undirected candidate graph was extracted after bipartite projection on the candidate-job bipartite graph using the minimum common application threshold. The duplicate pairs are reduced to unique pairs by taking their counts as the weight (number of the joint job application) of the link between the candidates with 1770479 pairs. A minimum weight threshold (min_weight ≥ 3) is applied in this operation. Due to the insufficient number of common applications, the weak links between the candidates are removed from the system after the projection operation and are not represented in the final graph. The properties of the candidate-candidate graph are shown in Table 2. Afterward, the experience information of each candidate represented on the final graph is collected as node features and scaled between 0 and 1, where 0 refers to the minimum years of experience and 1 refers to the maximum years of experience.

Table 2. Properties of homogeneous candidate-candidate graph.

Property	Value
Node Count	86560
Edge Count	1770479
Average Degree	20.453
Density	0.00047
Transitivity	0.601

The visualization of the candidate-candidate graph is shown in Figure 2. The purple areas represent commonly connected parts of the network where most of the applicants in these areas have similar job application behaviors.

3.2. Link Prediction

Link prediction is a widely explored subject in the analysis of graphs, and it holds significant importance in various applications. By identifying missing links within a graph, numerous problems can be addressed. For instance, in a graph representing connections between candidates, the prediction of links can facilitate the suggestion of candidates similar to his. Given its widespread popularity, extensive research has been conducted in this field.

The methods employed for link prediction can be categorized into three main groups. Firstly, traditional methods utilize a strategy [34] to compute a score for all potential links, i.e., pairs of nodes, that are absent in the network. Mathematical formulas based on network topologies, such as Adamic Adar and resource allocation, are used to calculate these scores. The score reflects the likelihood of a potential link, and the methods differ based on the strategies they employ. The scores are then ranked from highest to lowest, and a desired number of links are selected for prediction.

Secondly, similar to traditional methods, the scores for all missing links are calculated using various techniques. However, instead of simple ranking, machine learning models are employed [35]. In other words, a machine learning

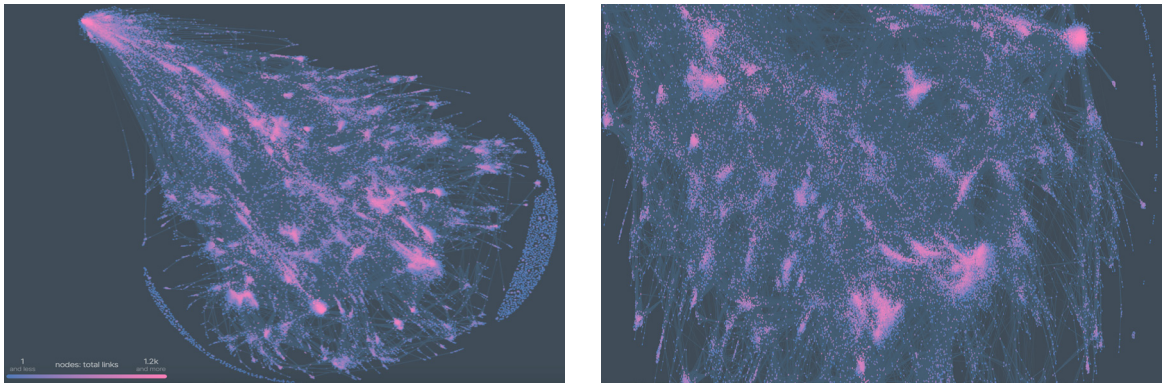

Fig. 2. Visualization of candidate-candidate graph on Cosmograph Visualizer.

model is constructed to predict links by considering different features of potential connections. These methods differ in terms of supervised learning techniques and the specific machine learning algorithms they utilize.

Lastly, recent studies have focused on graph embedding techniques [37, 36]. Researchers often develop new metrics or methods tailored to the requirements of their specific applications. These methods leverage different topological characteristics of the network. Graph neural network-based link prediction can be counted as an extension of this category. In our study, we focus on the foremost graph embedding techniques and graph convolutional networks to solve the link prediction problem of candidate-candidate networks.

3.2.1. Positive Negative Sampling

Positive Negative Sampling is a methodology that has been proposed with the objective of enhancing the accuracy of machine learning approaches.

Negative edges can be sampled randomly in two ways: globally or locally. In the global method, two nodes are selected, and if they are not connected, they are considered a negative sample. This process is repeated until a suitable negative sample is found. In the local method, a source node is randomly chosen, and negative edges are sampled using a Depth-First Search (DFS) at a certain distance from the source node, based on given probabilities.

There are two cases for sampling positive edges. In the first case, where graph connectivity must be preserved, the minimum spanning tree is initially calculated. The edges belonging to the minimum spanning tree are protected and cannot be sampled for the training set. The remaining edges, not part of the minimum spanning tree, are uniformly sampled at random until the desired number of positive edges is obtained.

In the second case, where the connectedness of the reduced graph is not guaranteed, positive edges are sampled uniformly at random from all edges in the graph, without considering whether they belong to the spanning tree (which is not calculated in this case).

Overall, the methods described allow for the sampling of negative and positive edges in graph datasets for training purposes, considering different requirements regarding graph connectivity.

3.2.2. node2vec

The node2vec node embedding has proven to be highly effective at extracting both local and global features of the graph. It can reflect both nearby nodes and the network's overall structure utilizing Breadth-first Sampling (BFS) and Depth-first Sampling (DFS) techniques using return parameter (p) and in-out parameter (q) as opposed to simply moving around the network randomly, which is how it differs from the ordinary random walk. In node2vec, the biased random walk briefly starts at the source node ($c_0 = \upsilon$) and moves across the subsequent nodes (x) in accordance with the distribution indicated in Equation 1.

$$P(c_i = x \mid c_{i-1} = \upsilon) = \begin{cases} \frac{\pi_{\upsilon x}}{Z} & \text{if } (\upsilon, x) \in E \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

The defined 2nd order random walk, first, traverses on the edge (t, υ) and then resides at node υ. Afterward, the walk decides on the next step while evaluating transition probabilities $\pi_{\upsilon x}$ on edges (υ, x) with the condition of the shortest path distance between nodes t and x denoted as d_{tx}. The search bias α calculation is shown in Equation 2 with two parameters p and q which guide the walk where $w_{\upsilon x}$ indicates the static edge weights between υ and x ($w_{\upsilon x} = 1$ for unweighted graphs).

$$\pi_{\upsilon x} = \alpha_{pq}(t,x) \cdot w_{\upsilon x}, \qquad where \qquad \alpha_{pq}(t,x) = \begin{cases} \frac{1}{p} \text{ if } d_{tx} = 0 \\ 1 \text{ if } d_{tx} = 1 \\ \frac{1}{q} \text{ if } d_{tx} = 2 \end{cases} \tag{2}$$

When walking on the network, p and q parameters determine whether the subsequent step is BFS or DFS. A low p-value (< min(q, 1)) guarantees that the next step will return to the referenced node, ensuring locality on the feature representation, whereas a large p-value (> max(q, 1)) specifies that the next step will primarily be in non-visited nodes from nearby neighbors (in case no other neighbors are present). In contrast, the q>1 condition guarantees that the next walk favors the nodes close to the currently visited node, and the q<1 condition ensures the farther nodes to be visited [25].

In the sequel of random walks, the generated feature vector of each node gets transformed into word embeddings using the word2vec algorithm.

3.2.3. GraphSAGE

GraphSAGE (SAmple and aggreGatE) is an inductive feature representation method for node embedding. Unlike standard embedding techniques, node embedding can be performed not only on the trained graph but also on the data that the model has not seen before. The embedding generation of the entire graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ aggregates information from the graph's farthest neighbors to their nearest neighbors at the chosen depth (k). As k increases, we start to have more topological information about the node's more distant neighbors. However, since this process requires feature aggregation for a large number of nodes, a randomly determined maximum of h neighbors are selected in each layer for the aggregation [26]. To learn useful, predictive representations in a completely unsupervised environment, a graph-based loss function is applied to the output representations, $\mathbf{z}_u, \forall u \in \mathcal{V}$, and the weight matrices $W^k, \forall k \in 1, ..., K$, and parameters of the aggregator functions via stochastic gradient descent. The graph-based loss function is shown in Equation 3 where υ is a node that co-occurs near u on a fixed-length random walk, σ is the sigmoid function, P_n is a negative sampling distribution, and Q defines the number of negative samples.

$$\mathcal{J}_{\mathcal{G}}(\mathbf{z}_u) = -log(\sigma(\mathbf{z}_u^\top \mathbf{z}_\upsilon)) - Q \cdot \mathbb{E}_{\upsilon_n \sim P_n(\upsilon)} log(\sigma(-\mathbf{z}_u^\top \mathbf{z}_{\upsilon_n})) \tag{3}$$

The mean aggregator and pooling aggregator are two different symmetrical aggregator functions that can be applied. The average of the feature vectors in each layer is taken and applied to the following layer in the mean aggregator. In the pooling aggregator, the feature vector of each neighbor is fed into a fully connected neural network, and the data obtained from the maximum pooling operation is aggregated to the upper layers. The mean aggregator was employed in the present study. The mean aggregator function equation is shown in Equation 4 where h^k denotes a node's representation at this step, h_υ^{k-1} denotes the node's previous layer representation, $\mathcal{N}(\upsilon)$ is a fixed-size, uniform samples from the set $u \in \mathcal{V} : (u, \upsilon) \in \mathcal{E}$. The parameters are then updated using the Stochastic Gradient Decent algorithm after the loss value for each computation graph is calculated.

$$\mathbf{h}_\upsilon^k \leftarrow \sigma(\mathbf{W} \cdot MEAN(\{\mathbf{h}_\upsilon^{k-1}\} \cup \{\mathbf{h}_u^{k-1}, \forall u \in \mathcal{N}(\upsilon)\}) \tag{4}$$

GraphSAGE's sample and aggregate approach are shown in Figure 3. An illustration of how the neighborhoods of k depth were sampled is shown in the first figure, an outline of feature aggregation in each layer is shown in the second figure, and the graph representation learning is shown in the third figure with the corresponding node labels in the embedding space (low-dimensional vector space).

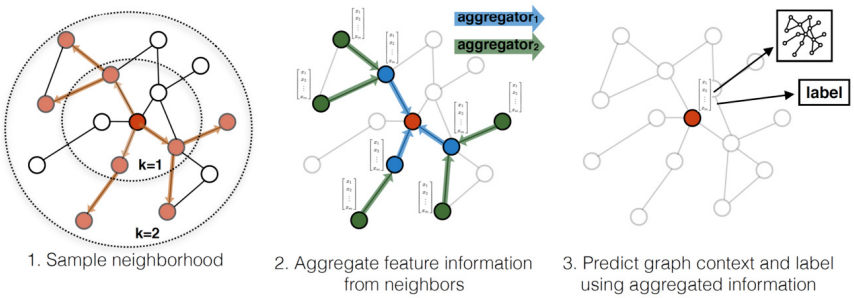

Fig. 3. Visual illustration of the GraphSAGE sample and aggregate approach, adopted from [26].

3.2.4. Graph Convolutional Network

The Graph Convolutional Neural Network (GCN) is a semi-supervised learning approach that employs a graph convolutional layer to apply a generalization of convolutions to non-Euclidean data [27][28]. In convolutional layers of Convolutional Neural Networks (CNN), the dot product (element-wise multiplication) of the regions where the filter intersects with the input data is calculated by hovering a filter over the input data. A similar operation is applied in GCNs, but since the data used is the graph data type, this operation is calculated over the adjacency matrix **A**. In an **A** adjacency matrix each node is shown as row and column indexes. If a node has neighboring nodes and is a weighted graph, the intersection cells of these two nodes are filled with the link weight value, otherwise filled with 1. To maintain the scale of the output feature network, each node is linked recursively on itself, and the **A** matrix is multiplied by the **D** matrix with diagonal node degrees, see Equation 5, left side of Figure 4.

$$H^{(i+1)} = \sigma((D^{1/2} \cdot A \cdot D^{1/2})H^i W^i) \tag{5}$$

By applying mean aggregation to the feature vectors formed as a result of this process, the feature representation of the graph is produced, see right side of Figure 4, where A is the adjacency matrix of the network, D is diagonal node degrees, H is the feature representation at level i, W is the weights at layer i.

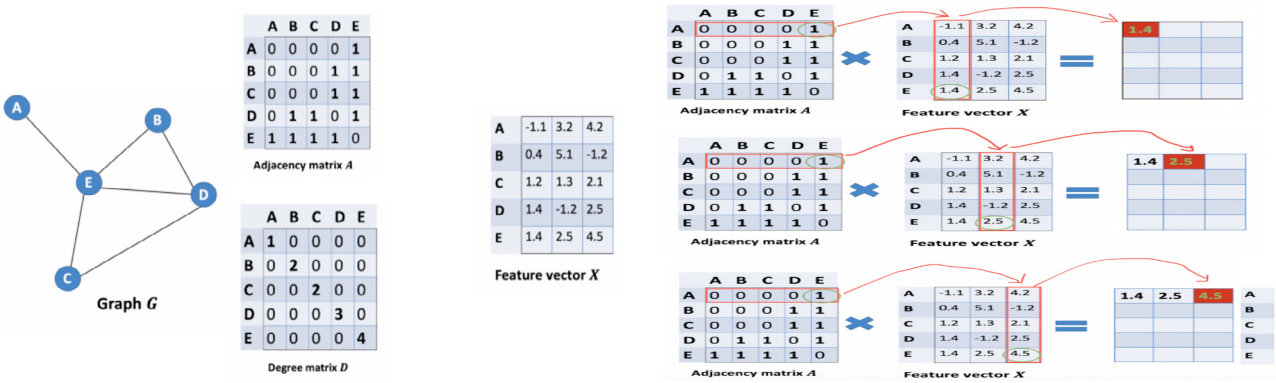

Fig. 4. The illustration of convolution and aggregation operations, adopted from [29].

As GCN can perform low dimensional feature representation of complex topological structures that CNN fails to represent, it is preferred for extracting feature representations of complex and arbitrary data sets [18].

4. Experimental Setup and Results

In order to compare the link prediction success of the proposed methods, the previously created candidate-candidate weighted graph is split into two separate graphs train_graph and test_graph.To accomplish this, the edges on the graph are taken as positive edges and the missing edges as negative edges, and a binary class structure is created, and the train test split process is then performed on these positive and negative samples. The weighted undirected candidate-candidate graph was used to construct the train and test graphs using the positive-negative sampling approach to train the node2vec, GraphSAGE, and GCN models.

The parameters for each algorithm specified in the Method section are shown in Table 3. With the given parameters, the node2vec, GraphSAGE, and GCN models are trained with the train_graph. The feature representations of the models are generated using Hadamard, L1, L2, and average binary operations. Since, the Logistic Regression Classifier is easy to implement, less prone to over-fitting, and outputs well-calibrated probabilities along with classification results, it is trained with each of the feature representation vectors of the training graph. The trained models are tested with the test_graph, and their link prediction success is measured with the AUC-ROC metric.

Table 3. The parameters that employed in the experimental setup.

Parameter	Value (node2vec, GraphSAGE, GCN)
Dimensionality of node embeddings (*dimension*)	128
length of each random walk (*walk_length*)	5
embedding model epoch number (*epochs*)	10
batch size of the embedding model (*batch_size*)	50
number of walks from each node (*walk_number*)	20, 1, 1

Table 4 shows the ROC-AUC score of node2vec, GraphSAGE, and GCN models with different types of binary operators applied to the embedding features. The link prediction performance of node2vec, GraphSAGE, and GCN with Hadamard, L1, L2, and average binary operations is evaluated on the test graph using the ROC-AUC score. GCN with L1 normalization produced the best result with a 0.94 ROC-AUC score. The highest scores are obtained with Hadamart operant for both GraphSAGE and node2vec with 0.90, and 0.84 respectively. As is shown in the last columns of Table 4 average operand produced the lowest scores for each of the models. Despite producing the highest scores, training the GCN model took four times longer than the GraphSAGE and two times longer than node2vec. Therefore, it would be advantageous to prefer GraphSAGE, which produces the second-highest score, over GCN in real-time applications.

Table 4. ROC-AUC scores of trained models with *Hadamard*, L_1, L_2, *average* binary operands.

Embedding Method	Binary Operator			
	Hadamard	L_1	L_2	*average*
node2vec	**0.840927**	0.812559	0.821498	0.639340
GraphSAGE	**0.896028**	0.889471	0.890269	0.727795
GCN	0.897249	**0.941469**	0.920184	0.884374

5. Conclusion and Future Work

In this paper, we presented a graph-based candidate recommendation system for employers on online recruitment platforms. Using job seekers' job application behavior on the website, we were able to find the relations between job seekers. The proposed feature representation models were trained on the candidate application data which enables us to perform link prediction tasks to find qualified candidates for the open positions.

Because of the volume of job applications on Kariyer.Net, the collected data size may reach very high numbers. Trials on 30-day (1 month), 21-day (3 weeks), and 14-day (2 weeks) data sets were conducted; however, only the 10 days of application data could be used for the implementation of this study since there aren't enough hardware features to handle such high dimensional data. Since the life cycle of the jobs posted on Kariyer.Net is either 30 or 60 days, it is anticipated that the Job Seeker (candidate) graph, which is dynamically supplied over 30 days, will yield more successful outcomes.

In the future, a graph-based and content-based hybrid recommendation system can be developed to enhance the recommendation results with the qualifications of the candidates.

References

[1] Vijayalakshmi, M. "E-Recruitment Challenges" (2013) *International Journal Of Social Science & Interdisciplinary Research*. **2**, 1-4

[2] Hada, B. & Gairola, S. (2015) "Opportunities & challenges of E-recruitment" *Journal Of Management Engineering And Information Technology*. **2**, 1-4

[3] Ugo Chuks Okolie1, I. (2017) "E-Recruitment: Practices, Opportunities and Challenges" *European Journal Of Business And Management*. **9**, https://core.ac.uk/works/70299525

[4] Shalaby, W., AlAila, B., Korayem, M., Pournajaf, L., Aljadda, K., Quinn, S. & Zadrozny, W. (2017) "Help me find a job: A graph-based approach for job recommendation at scale."

[5] Dave, V., Zhang, B., Al Hasan, M., AlJadda, K. & Korayem, M. (2018) "A Combined Representation Learning Approach for Better Job and Skill Recommendation" *Proceedings Of The 27th ACM International Conference On Information And Knowledge Management*. pp. 1997-2005, https://doi.org/10.1145/3269206.3272023

[6] Yilmaz, E., Balcisoy, S. & Bozkaya, B. (2022) "A Link Prediction-based Recommendation System using Transactional Data"

[7] Dhamija, P. (2012) "E-recruitment: a roadmap towards e-human resource management" *Researchers World*. **3**, 33

[8] Rafter, R., Bradley, K. & Smyth, B. (2000) "Personalised retrieval for online recruitment services" *The BCS/IRSG 22nd Annual Colloquium On Information Retrieval (IRSG 2000), Cambridge, UK, 5-7 April, 2000*.

[9] Carrer-Neto, W., Hernandez-Alcaraz, M., Valencia-Garcia, R. & Garcia-Sanchez, F. (2012) "Social knowledge-based recommender system. Application to the movies domain" *Expert Systems With Applications*. **39**, 10990-11000

[10] Lee, S., Cho, Y. & Kim, S. (2010) "Collaborative filtering with ordinal scale-based implicit ratings for mobile music recommendations" *Information Sciences*. **180**, 2142-2155

[11] Huang, Z., Zeng, D. & Chen, H. (2007) "A comparison of collaborative-filtering recommendation algorithms for e-commerce" *IEEE Intelligent Systems*. **22**, 68-78

[12] Yu, Z., Zhou, X., Hao, Y. & Gu, J. (2006) "TV program recommendation for multiple viewers based on user profile merging" *User Modeling And User-adapted Interaction*. **16** pp. 63-82

[13] Gutiérrez, F., Charleer, S., De Croon, R., Htun, N., Goetschalckx, G. & Verbert, K. (2019) "Explaining and Exploring Job Recommendations: A User-Driven Approach for Interacting with Knowledge-Based Job Recommender Systems" *Proceedings Of The 13th ACM Conference On Recommender Systems*. pp. 60-68, https://doi.org/10.1145/3298689.3347001

[14] Paparrizos, I., Cambazoglu, B. & Gionis, A. (2011) "Machine Learned Job Recommendation" *Proceedings Of The Fifth ACM Conference On Recommender Systems*. pp. 325-328, https://doi.org/10.1145/2043932.2043994

[15] Kille, B., Abel, F., Hidasi, B. & Albayrak, S. (2015) "Using interaction signals for job recommendations" *Mobile Computing, Applications, And Services: 7th International Conference, MobiCASE 2015, Berlin, Germany, November 12–13, 2015, Revised Selected Papers 7*. pp. 301-308

[16] Xiang, T., Yang, X., Zeng, S., Liu, Z. & Huang, Z. (2020) "Comparison between Knowledge Network System and Collaborative Filtering in Recommender System" *Proceedings Of The 8th International Conference On Computer And Communications Management*. pp. 47-53, https://doi.org/10.1145/3411174.3411177

[17] Bobadilla, J., Ortega, F., Hernando, A. & Gutierrez (2013) "A Recommender Systems Survey" *Knowledge-based Systems*, **46** 109-132.

[18] Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C. and Philip, S.Y. (2020) "A comprehensive survey on graph neural networks" IEEE transactions on neural networks and learning systems, 32(1), pp.4-24.

[19] Nagy, F., Haroun, A., Abdel-Kader, H. & Keshk, A. (2021) "A review for recommender system models and deep learning. *IJCI. International Journal Of Computers And Information*" **8**, 170-176

[20] Jang, S., Kim, S. & Ha, J. (2007) "Graph-based recommendation systems: Comparison analysis between traditional clustering techniques and neural embedding" *Tech. Rep*.

[21] Aggarwal, C., Wolf, J., Wu, K. & Yu, P. (1999) "Horting Hatches an Egg: A New Graph-Theoretic Approach to Collaborative Filtering" *Proceedings Of The Fifth ACM SIGKDD International Conference On Knowledge Discovery And Data Mining*. pp. 201-212, https://doi.org/10.1145/312129.312230

[22] Ravita, R. & Rathi, S. (2022) "Inductive Learning Approach in Job Recommendation" *International Journal Of Intelligent Systems And Applications In Engineering*. **10**, 242-251, https://ijisae.org/index.php/IJISAE/article/view/1829

[23] Park, M. & Geum, Y. (2022) "Two-stage technology opportunity discovery for firm-level decision making: GCN-based link-prediction approach" *Technological Forecasting And Social Change*. **183** pp. 121934

[24] Sivaramakrishnan N. (2018) "Validating Effective Resume Based on Employer's Interest With Recommendation System" *International Journal Of Pure And Applied Mathematics*. **119**, https://core.ac.uk/works/70299525

[25] Grover, A. & Leskovec, J. (2016) “node2vec: Scalable feature learning for networks" *Proceedings Of The 22nd ACM SIGKDD International Conference On Knowledge Discovery And Data Mining*. pp. 855-864
[26] Hamilton, W., Ying, Z. & Leskovec, J. (2017) “Inductive representation learning on large graphs" *Advances In Neural Information Processing Systems*. **30**
[27] Kipf, T. & Welling, M. (2017) Semi-Supervised Classification with Graph Convolutional Networks"
[28] Defferrard, M., Bresson, X. & Vandergheynst, P. (2017) “Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering"
[29] Pham, C. Graph Convolutional Networks (GCN). (2020), https://www.topbots.com/graph-convolutional-networks/
[30] Kara, A., Daniş, F., Orman, G., Turhan, S. & Özlü, Ö. Job Recommendation Based on Extracted Skill Embeddings. *Intelligent Systems And Applications*. pp. 497-507 (2023)
[31] Kara, A., Daniş, F., Orman, G. & Turhan, S. Extracting Relations Between Sectors. (2022)
[32] Özlü, Ö., Orman, G., Daniş, F., Turhan, S., Kara, K. & Yücel, T. Similarity-Based Résumé Matching via Triplet Loss with BERT Models. *Intelligent Systems And Applications*. pp. 520-532 (2023)
[33] Keskin, Ş., Balı, Y., Orman, G., Daniş, F. & Turhan, S. Determining Column Numbers in Résumés with Clustering. *Artificial Intelligence Applications And Innovations*. pp. 460-471 (2022)
[34] Lü, L., & Zhou, T. Link prediction in complex networks: A survey. *Physica A: statistical mechanics and its applications*, 390(6), pp.1150-1170 (2011).
[35] Malhotra, D., & Goyal, R. Supervised-learning link prediction in single layer and multiplex networks. *Machine Learning with Applications*, pp.6, 100086 (2021).
[36] Rossi, A., Barbosa, D., Firmani, D., Matinata, A., & Merialdo, P. Knowledge graph embedding for link prediction: A comparative analysis. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 15(2), pp. 1-49 (2021).
[37] Wang, M., Qiu, L., & Wang, X. A survey on knowledge graph embeddings for link prediction. *Symmetry*, 13(3), pp.485 (2021).